

如何从XTP行情迁移到XTPX行情

目录

- [如何从XTP行情迁移到XTPX行情](#)
 - [1. 文件修改](#)
 - [2. 代码修改](#)

1. 文件修改

- 从官网上下载XTPX版本的SDK，将XTP版本行情的所有库文件和头文件全部替换成XTPX版本行情的库文件和头文件，步骤如下：
 - (1) 把XTP API行情使用的以下头文件和库文件删除：

-  libxtpquoteapi.so
-  xquote_api_rebuild_tbt_struct.h
-  xquote_api_struct.h
-  xtp_quote_api.h
-  xtpquoteapi.dll
-  xtpquoteapi.lib

- (2) 添加XTPX API行情的以下头文件和库文件：

-  xquote_x_api_data_type.h
-  xquote_x_api_rebuild_tbt_struct.h
-  xquote_x_api_struct.h
-  xtpx_api_data_type.h
-  xtpx_api_struct_common.h
-  xtpx_quote_api.h
-  xtpxquoteapi.dll
-  xtpxquoteapi.lib
-  libxtpxquoteapi.so

- (3) 将头文件xtp_api_struct.h里的行情相关的头文件引用全部去掉或者备注掉，如下图所示：

```

////////////////////////////////////
// @author 中泰证券股份有限公司
// @file xtp_api_struct.h
// @brief 定义业务数据结构
////////////////////////////////////
#ifndef _XTP_API_STRUCT_H_
#define _XTP_API_STRUCT_H_

#include "xtp_api_struct_common.h"
// #include "xquote_api_struct.h"
// #include "xoms_api_struct.h"
#include "xoms_api_fund_struct.h"
// #include "xquote_api_rebuild_tbt_struct.h"

#endif // !_XTP_API_STRUCT_H_

```

2. 代码修改

- XTPX版本的行情所有接口类和数据类型都定义在域名XTPX::API之下，客户在使用相关类和参数类型时，请注意使用域名XTPX::API。另外，XTPX版本的行情与XTP版本的交易一起使用时，如编译遇到参数类型不明确的错误时，那么交易相关报错的接口参数类型前加::，行情相关报错的接口参数类型前加XTPX::API::，从而解决参数类型不明确的编译报错，代码示例如下：

- (1) 行情接口调用示例

```

//创建行情api
XTPX::API::QuoteApi* pQuoteApi =
XTPX::API::QuoteApi::CreateQuoteApi(client_id, filepath.c_str(),
XTPX::API::XTP_LOG_LEVEL_DEBUG); //log日志级别可以调整

```

- (2) 交易接口调用示例

```

//创建交易类Api
XTP::API::TraderApi* pUserApi =
XTP::API::TraderApi::CreateTraderApi(client_id, filepath.c_str(),
::XTP_LOG_LEVEL_DEBUG);

```

- XTPX版本的QuoteApi在登录udp连接方式的行情服务器之前，必须先调用接口SetConfigFile()设置正确的配置文件。用户在订阅行情之前，需要将订阅的行情，在配置文件里设置成打开标识，再调用订阅接口才行，否则缺一将无法收到行情，例如想要订阅L2沪市和深市逐笔行情，那么在配置文件中如下参数需设置成ON：

```

[subscribe_quote_type]
.....
sh_level2_tbt_stock = ON
sh_level2_tbt_bond = ON
sz_level2_tbt_stock = ON
sz_level2_tbt_bond = ON
.....

```

- 在使用XTPX版本时去除相关接口SetUDPBufferSize, SetUDPRecvThreadAffinity, SetUDPRecvThreadAffinityArray, SetUDPParseThreadAffinity, SetUDPParseThreadAffinityArray调用,改为在配置文件里设置。如订阅了快照和逐笔行情,那么快照、逐笔的缓存和线程绑核参数可以设置如下:

```
[md]
decode_flag = 1  #1表示解码的快照数据, 目前api提供的只有解码的行情数据
parse_cpu_id = 2 #解析线程绑核的cpu核id(逻辑核), 0表示不绑核
[md.normal]
enable = ON  #ON 表示启用软件行情的快照, OFF表示不启用
recv_cpu_id = 3  #接收线程绑核的cpu核id(逻辑核), 0表示不绑核
L1_buf_capacity = 256 #一级缓存的大小, 最小为256k个缓存单元
L2_buf_capacity = 8  #二级缓存的大小, 最小为8k个缓存单元
.....
[md.fpga]
enable = OFF #ON表示启用硬件行情的快照, OFF表示不启用
recv_cpu_id = 3 #接收线程绑核的cpu核id(逻辑核), 0表示不绑核
L1_buf_capacity = 256 #一级缓存的大小, 最小为256k个缓存单元
L2_buf_capacity = 8  #二级缓存的大小, 最小为8k个缓存单元
.....
[tbt]
decode_flag = 1  #1表示解码的逐笔数据, 目前api提供的只有解码的行情数据
parse_cpu_id = 4 #解析线程绑核的cpu核id(逻辑核), 0表示不绑核
[tbt.normal]
enable = ON  #ON表示启用软件行情的逐笔, OFF表示不启用
recv_cpu_id = 5  #接收线程绑核的cpu核id(逻辑核), 0表示不绑核
L1_buf_capacity = 256 #一级缓存的大小, 最小为256k个缓存单元
L2_buf_capacity = 8  #二级缓存的大小, 最小为8k个缓存单元
.....
[tbt.fpga]
enable = OFF  #ON表示启用硬件行情的逐笔, OFF表示不启用
recv_cpu_id = 5 #接收线程绑核的cpu核id(逻辑核), 0表示不绑核
L1_buf_capacity = 256 #一级缓存的大小, 最小为256k个缓存单元
L2_buf_capacity = 8  #二级缓存的大小, 最小为8k个缓存单元
.....
```

- 在XTP版本中如果有使用快照回调接口OnDepthMarketData()中的参数XTP_MARKETDATA_TYPE data_type来判断union是哪个数据类型的话, 在XTPX版本要更改为参数XTP_MARKETDATA_TYPE_V2 data_type_v2来判断。示例代码如下:

```
void MyQuoteSpi::OnDepthMarketData(XTPMD * market_data, int64_t bid1_qty[],
int32_t bid1_count, int32_t max_bid1_count, int64_t ask1_qty[], int32_t
ask1_count, int32_t max_ask1_count)
{
    if (market_data->data_type_v2 == XTP_MARKETDATA_V2_INDEX) //指数
    {
        //代码处理
    }
}
```

```

    }
    else if (market_data->data_type_v2 == XTP_MARKETDATA_V2_OPTION)//期权
    {
        //代码处理
    }
    else if (market_data->data_type_v2 == XTP_MARKETDATA_V2_ACTUAL)//现货(股票/基金
等)
    {
        //代码处理
    }
    else if (market_data->data_type_v2 == XTP_MARKETDATA_V2_BOND)//债券
    {
        //代码处理
    }
}

```

- 在XTP版本中如果有使用快照回调接口OnDepthMarketData()中的参数ticker_status来判断当前ticker的交易状态的话，请根据XTPX版本中新的标志位进行判断，标志位如下：

沪市如下：

第0位：

- 'S',启动（开市前）时段
- 'C',开盘集合竞价时段
- 'T',连续竞价时段
- 'E',闭市时段
- 'P',产品停牌
- 'M',表示可恢复交易的熔断时段（盘中集合竞价）
- 'N',表示不可恢复交易的熔断时段（暂停交易至闭市）
- 'U',表示收盘集合竞价时段

第1位：

- '0',此产品不可正常交易
- '1',此产品可以正常交易
- 无意义填充格

第2位：

- '0',未上市
- '1',已上市

第3位：

- '0',此产品在当前时段，不接受进行新订单申报
- '1',此产品在当前时段，可接受进行新订单申报
- 无意义填充格

深市如下：

第0位：

- 'S',启动（开市前）时段
- 'O',开盘集合竞价时段
- 'T',连续竞价时段
- 'B',休市
- 'C',表示收盘集合竞价时段

- 'E', 闭市
- 'H', 临时停牌
- 'A', 盘后交易
- 'V', 波动性中断

第1位：

- '0', 正常状态
- '1', 全天停牌

- 在XTPX版本中行情回补无需登录回补服务器，若行情数据有丢失，直接调用接口 `RequestRebuildQuote()` 请求回补行情。所以在3.0版本中如果有使用回补行情相关接口 `LoginToRebuildQuoteServer()`、`LogoutFromRebuildQuoteServer()` 以及继承接口类 `quotespi` 中的回调接口 `OnRebuildQuoteServerDisconnected()` 的话，在使用XTPX版本时都得去除。
- XTPX版本不再使用接口 `SetUDPSeqLogOutPutFlag()` 来控制异步日志输出，可通过参数在接口 `CreateQuoteApi()` 里进行设置，所以如果代码中有调用接口 `SetUDPSeqLogOutPutFlag()` 的话，请修改成通过接口 `CreateQuoteApi()` 里的参数来控制异步日志输出。
- XTPX版本不再使用接口 `GetTradingDay()`，如在XTP版本的行情程序中有调用 `GetTradingDay()` 来获取当前交易日，在使用XTPX版本时得去除。客户在订阅行情数据时，行情数据中就有当前交易日的时间戳。